

JEON MINGYU · BACKEND ENGINEER

트래픽과 장애 속에서도 데이터의 의미를 지키는 백엔드 엔지니어링

LoL 경기 이벤트 기반 선착순 쿠폰의 동시성 제어, 실제 쿠폰 생성, 실시간 재고,
Redis 장애 복구와 대규모 데이터 검증을 설계하고 구현했습니다.

Java 21

Spring Boot

Redis Lua

MySQL

Kafka Outbox

SSE

k6

NAME

전민규

ROLE

Backend Engineer

GITHUB

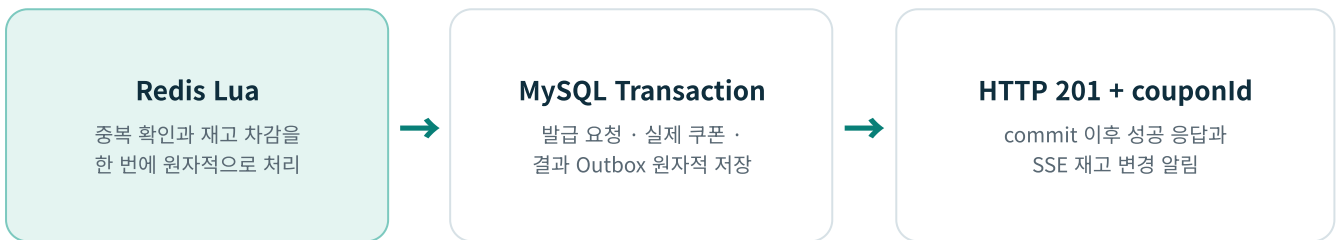
github.com/minggure

FEATURED PROJECT · CLUTCH-BE

Redis 성공이 아니라 실제 쿠폰 생성을 사용자 성공으로 정의했습니다

Redis 재고 차감만 성공한 뒤 DB 저장이 실패하면 사용자는 쿠폰이 없는 성공 응답을 받습니다.

MySQL에 실제 `user_coupon` 이 생성된 시점을 최종 성공으로 정의하고 전체 흐름을 재설계했습니다.



1 초과·중복 발급 차단

Redis Lua에서 재고와 회차별 당첨자 기록을 함께 변경해 경쟁 조건을 차단했습니다.

2 실패 보상

DB rollback 시 Redis 재고와 당첨 기록을 되돌려 다음 회차 재참여 가능 상태를 복구했습니다.

3 후속 전달 분리

Kafka를 발급의 필수 경로에서 분리하고 Outbox로 후속 결과 전달의 재시도를 담당하게 했습니다.

담당 범위

- 쿠폰 이벤트·회차·발급 요청 API와 연속성
- Redis 재고 초기화와 Lua 동시성 제어
- 동기 쿠폰 생성과 rollback 보상
- Kafka 계약, Outbox 저장·발행·재시도
- 잔여 재고 API와 SSE 스트림
- Redis 자동 장애 복구와 관리자 API
- 고부하 병목 분석과 Redis 선평단
- 대량 데이터 정합성 검증과 후속 개선

빠른 응답보다 먼저 지킨 기준은 “사용자가 받은 성공 응답은 실제 쿠폰을 보장한다”였습니다.

프로젝트 · github.com/seok-cess/Clutch-BE

근거 · Git 작성 이력과 ADR-001 실제 쿠폰 생성 기준의 동기 발급

DESIGN FOR FAILURE

Redis가 비어 있어도 초과 발급하지 않도록

정확한 재고를 알 수 없을 때 MySQL로 우회 발급하면 두 개의 동시성 경로가 생깁니다. 발급을 중단하고 신뢰 가능한 원본에서 복구하는 **Fail Closed** 정책을 선택했습니다.

- UNAVAILABLE · 발급 차단**
 연결 장애나 필수 키 누락을 감지하면 새 요청을 HTTP 503으로 종료합니다.
- RECOVERING · 정합성 검증**
 성공 요청, 실제 쿠폰 수와 사용자 집합이 일치하는지 먼저 확인합니다.
- 재고·당첨자·컨텍스트 재구축**
 MySQL의 실제 쿠폰을 기준으로 Redis 데이터를 Lua로 함께 교체합니다.
- READY · 결과 재검증 후 재개**
 기대값과 같을 때만 발급을 재개하고 불일치 시 자동 복구를 중단합니다.

100장 장애 실험

20장 발급 → Redis 중단 → 장애 중 503 → 자동 복구
→ 남은 80장 발급 → 101번째 요청은 정상 품질

최종 결과

실제 쿠폰 100장

초과 발급 0건 · 중복 발급 0건

공통 성공 수량 행의 X-lock 병목

19/20

Hikari active

178

커넥션 최대 대기

2,914

행 잠금 횟수 / 분

1,115s

잠금 대기 시간 / 분

BEFORE · 모든 성공 요청

- 실제 쿠폰 저장
- 동일 `success_count` 행 갱신
- X-lock 직렬 대기
- Hikari와 Tomcat으로 적체 확산



AFTER · 책임 분리

- 발급 transaction은 실제 쿠폰만 저장
- 5초 주기 단일 GROUP BY 집계
- 전용 인덱스로 커버링 집계
- MySQL named lock으로 중복 실행 차단

성능 수치를 다룬 방식

20,000 VU 시험에서 실제 쿠폰 10,000건을 저장했지만 TCP 연결 실패와 구버전 컨테이너 실행이 섞였습니다. 이를 서버 최종 처리량으로 주장하지 않고 최신 배포·분산 부하 발생기·네트워크 구간을 분리할 재검증 조건으로 기록했습니다.

VERIFY, THEN CHANGE

100만 사용자·351만 요청 이력을 안전하게 검증했습니다

대량 데이터는 수정 없이 **REPEATABLE READ** 일관 스냅샷과 **READ ONLY transaction**으로
검사했습니다. 개인정보는 출력하지 않고 건수·위반 수·fingerprint만 남겼습니다.

1,000,003

사용자

3,517,209

쿠폰 발급 요청

797,209

실제 발급 쿠폰

핵심 위반 검사 · 0건

✓ 사용자·이벤트·회차·항목 고아 참조

✓ 요청과 실제 쿠폰의 관계 불일치

✓ 성공 요청 쿠폰 누락·실패 요청 쿠폰 연결

✓ 동일 사용자·회차 중복 요청 및 중복 쿠폰

✓ 수량 초과 발급과 잘못된 재고 범위

✓ 상태와 완료·사용·취소 시각 불일치

✓ 장시간 PENDING과 만료 후 OPEN 회차

✓ 성공 집계와 실제 쿠폰 수 불일치

검증이 찾은 상태 계약 차이

DB의 CANCELLED와 Java enum이 달랐고 만료 판정도
경로마다 달랐습니다.

조치: 상태 계약 보완, 만료 판정 일원화, 만료 후 사용·
취소 차단과 회귀 테스트

검증이 찾은 집계 비용

이벤트 항목 기준 선두 인덱스가 없어 데이터 증가 시
반복 전체 탐색 위험이 있었습니다.

조치: Flyway V16 인덱스, 단일 GROUP BY, 실행
계획의 Using index 확인

테스트 통과에서 끝내지 않고, 대량 데이터가 드러낸 WARN을 코드·인덱스·상태 계약 개선으로
달았습니다.

Engineering Stack

Java 21

Spring Boot

Spring Data JPA

MySQL 8

Flyway

Redis

Lua

Kafka

Outbox

SSE

Micrometer

JUnit

Spring Test

k6

Docker Compose

GitHub · <https://github.com/minggyure>

작성 기준 · 저장소 Git 이력과 확정된 ADR·도메인·검증 문서 · 2026-08-28